

ДЗ 1 БД

Бизнес-домен:

Вы разрабатываете базу данных для платформы онлайн-образования. Платформа позволяет пользователям регистрироваться, выбирать курсы и проходить обучение.

Каждый курс состоит из нескольких уроков. Преподаватели могут создавать курсы и получать отчисления от продаж. Студенты могут оставлять отзывы на курсы и оценивать их по шкале от 1 до 5.

Задание 1. Концептуальная модель

5 основных сущностей

- User
- Course
- Lesson
- Enrollment
- Review

Обоснование

User(пользователь)

Бизнес-требование: «Пользователи регистрируются»

Обоснование выбора: Данная сущность необходима, чтобы знать кто заходит на сайт. Отдельная таблица нужна, чтобы человек мог один раз зарегистрироваться и войти под своим логином и паролем. Без этого мы не поймем, кто из них студент, а кто преподаватель.

Course(курс)

Бизнес-требование: «Создание курсов преподавателями для студентов»

Обоснование выбора: Курс — это главный «товар» на сайте. Отдельная таблица нужна, чтобы хранить его название, описание и цену в одном месте. Если что-то в описании поменяется, мы исправим это в одной строчке, а не во всей базе.

Lesson(урок)

Бизнес-требование: «Каждый курс состоит из уроков»

Обоснование выбора: Курс состоит из нескольких частей, поэтому его нельзя записать в одну строку. Мы выносим уроки в отдельную таблицу, чтобы для каждого хранить его номер по порядку, тему и ссылку на видео.

Enrollment(запись на курс)

Бизнес-требование: «Выбор и прохождение курсов»

Обоснование выбора: Эта таблица связывает студента с курсом и запоминает: когда он купил курс, сколько заплатил и прошел ли его до конца.

Review(отзыв)

Бизнес-требование: «Студенты могут оставлять отзывы на курсы и оценивать их по шкале от 1 до 5»

Обоснование выбора: Нам нужна отдельная таблица, чтобы сохранять текст отзыва и оценку от студента. Это позволяет показывать отзывы на странице курса, считать его средний рейтинг и следить, чтобы один студент не оставлял больше одного отзыва.

Связи между ними

USER 1 – N Course (Преподаватель создает курсы)

User (1) – (N) Enrollment (Студент может записаться на несколько курсов)

User (1) – (N) Review (Студент оставляет отзывы на курсы)

Course (1) – (N) Lesson (Курс содержит несколько уроков)

Course (1) – Review (N) (На один курс может быть много отзывов)

Course (1) – (N) Enrollment (На один курс может быть много записей)

Задание 2. Логическая модель

Описание сущностей и их атрибутов

Как бизнес-требования повлияли на выбор атрибутов: users.role появился из требования «нужно разграничивать студентов и преподавателей»; courses.price — из требования «преподаватели получают отчисления от продаж» (нужна сумма, от которой считать выплату); lessons.order_number — из требования «курс состоит из нескольких уроков», которые должны идти по порядку; enrollments.enrolled_at и status — из требования «студенты выбирают и проходят курсы» (нужно фиксировать факт и статус прохождения); reviews.rating и comment — напрямую из требования «студенты оставляют отзывы и оценивают курсы по шкале от 1 до 5».

User

- **user_id** (INTEGER, Primary Key) — Уникальный идентификатор пользователя.
- **name** (VARCHAR(100)) — Имя и фамилия пользователя.
- **email** (VARCHAR(255), Unique, Not Null) — Электронная почта.
- **password** (VARCHAR(255), Not Null) — Хэш пароля пользователя.
- **role** (VARCHAR(20), Not Null) — Роль в системе ('student' или 'teacher') для разграничения прав.

Course

- **course_id** (INTEGER, Primary Key) — Уникальный идентификатор курса.
- **teacher_id** (INTEGER, Foreign Key User.user_id) — Ссылка на преподавателя, создавшего курс.
- **name_course** (VARCHAR(200), Not Null) — Название курса.
- **price** (DECIMAL(10, 2), Not Null) — Стоимость курса.
- **count_lesson** (INTEGER, Default 0) — Количество уроков в курсе (агрегируемый атрибут).

Lesson

- **lesson_id** (INTEGER, Primary Key) — Уникальный идентификатор урока.
- **course_id** (INTEGER, Foreign Key Course.course_id) — Ссылка на курс, к которому относится урок.
- **name_lesson** (VARCHAR(200), Not Null) — Название урока.
- **lesson_time** (INTEGER) — Длительность урока (в минутах).
- **order_number** (INTEGER, Not Null) — Порядковый номер урока внутри курса.

Enrollment

- **enrollment_id** (INTEGER, Primary Key) — Уникальный номер записи на курс.
- **user_id** (INTEGER, Foreign Key User.user_id) — Ссылка на записавшегося студента.
- **course_id** (INTEGER, Foreign Key Course.course_id) — Ссылка на купленный курс.
- **enrolled_at** (TIMESTAMP, Not Null) — Дата и время записи на курс.
- **status** (VARCHAR(20), Not Null) — Статус прохождения ('active', 'completed').

Review

- **review_id** (INTEGER, Primary Key) — Уникальный идентификатор отзыва.
- **user_id** (INTEGER, Foreign Key User.user_id) — Ссылка на автора отзыва (студента).
- **course_id** (INTEGER, Foreign Key Course.course_id) — Ссылка на оцениваемый курс.
- **rating** (INTEGER, Not Null) — Оценка курса по шкале от 1 до 5.
- **comment** (TEXT) — Текст отзыва.
- **created_at** (TIMESTAMP, Not Null) — Дата публикации отзыва.

Кардинальность связей

USER 1 - N Course

Со стороны User: Один преподаватель может создать или несколько курсов (1 к N).

Со стороны Course: Каждый курс обязательно создается строго одним преподавателем (N к 1).

User (1) – (N) Enrollment

Со стороны User: Студент может сделать 0 или несколько записей на курсы (1 к N).

Со стороны Enrollment: Каждая запись принадлежит строго одному пользователю (N к 1).

User (1) – (N) Review

Со стороны User: Пользователь может написать от до нескольких отзывов на разные курсы (1 к N).

Со стороны Review: Каждый отзыв принадлежит конкретному пользователю.

Course (1) – (N) Lesson

Со стороны Course: Курс может содержать 0 или несколько уроков (1 к N).

Со стороны Lesson: Урок принадлежит строго одному конкретному курсу (N к 1).

Course (1) – Review (N)

Со стороны Course: Курс может иметь от до множества отзывов (1 к N).

Со стороны Review: Каждый отзыв относится к конкретному курсу.

Course (1) – (N) Enrollment

Со стороны Course: На курс может записаться 0 или много студентов (1 к N).

Со стороны Enrollment: Каждая запись относится строго к одному курсу (N к 1).

Неидентифицирующая

Классификация связей

Примечание: связь классифицируется как идентифицирующая по смысловой (бизнес) зависимости — существует ли дочерняя запись без родителя, — а не по структуре первичного ключа. На уровне схемы все таблицы используют собственные суррогатные SERIAL-ключи, поэтому формально (по составу PK) все связи в физической модели неидентифицирующие.

User (teacher) → Course

Неидентифицирующая

Курс – самостоятельная сущность: у него своё название, цена, содержание. Смысл курса не теряется, если его переназначить другому преподавателю — курс не перестаёт быть тем же курсом.

Course → Lesson

Идентифицирующая

Урок вне курса не имеет смысла – это часть курса, а не отдельный объект.

User (student) → Enrollment

Идентифицирующая

Enrollment – это когда конкретный студент записался на конкретный курс. Без студента запись не существует и не имеет смысла сама по себе.

User (student) → Review

Идентифицирующая

Отзыв не существует без автора – это оценка конкретного студента.

Course → Review

Идентифицирующая

Отзыв не существует без курса, к которому он относится.

Course → Enrollment

Неидентифицирующая

Enrollment имеет собственный суррогатный ключ enrollment_id, не включающий course_id в свой состав, поэтому формально связь неидентифицирующая — хотя семантически запись без курса не имеет смысла.

Задание 3. Физическая модель

Как бизнес-требования повлияли на выбор ограничений

- users.role: NOT NULL + CHECK (role IN ('student', 'teacher')) — по требованию «нужно разграничивать студентов и преподавателей» это должно проверяться на уровне базы.
- users.email: UNIQUE NOT NULL — при регистрации один email соответствует ровно одному аккаунту.
- courses.price: тип DECIMAL(10, 2) для точных денежных сумм (не FLOAT, который округляет неточно) и CHECK (price >= 0), так как курс платный и отрицательная цена невозможна.
- lessons.order_number: CHECK (order_number > 0) и UNIQUE (course_id, order_number) — по требованию «уроки идут по порядку внутри курса» два урока одного курса не могут иметь одинаковый номер.
- enrollments.status: CHECK (status IN ('active', 'completed')) — отражает статусы прохождения курса.

- enrollments: UNIQUE (user_id, course_id) — повторная запись студента на тот же курс не нужна.
- reviews.rating: CHECK (rating BETWEEN 1 AND 5) — прямое отражение требования «оценивать курсы по шкале от 1 до 5».
- reviews: UNIQUE (user_id, course_id) — один студент не должен оставлять больше одного отзыва на курс.
- Внешние ключи: courses → users — ON DELETE RESTRICT (неидентифицирующая связь: при удалении преподавателя курсы и история продаж не должны пропадать). Остальные связи (lessons → courses, enrollments → users/courses, reviews → users/courses) — ON DELETE CASCADE, так как в Задании 2 они отмечены как идентифицирующие: дочерняя запись не имеет смысла без родителя.
- На все внешние ключи добавлены индексы, так как по частым запросам регулярно требуется выборка по курсу/студенту — без индекса такие запросы будут сканировать всю таблицу.

SQL-скрипт (PostgreSQL)

```
DROP TABLE IF EXISTS reviews CASCADE;
DROP TABLE IF EXISTS enrollments CASCADE;
DROP TABLE IF EXISTS lessons CASCADE;
DROP TABLE IF EXISTS courses CASCADE;
DROP TABLE IF EXISTS users CASCADE;
```

```
CREATE TABLE users (
  user_id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(255) UNIQUE NOT NULL,
  password VARCHAR(255) NOT NULL,
  role VARCHAR(20) NOT NULL
    CHECK (role IN ('student', 'teacher'))
);
```

```
CREATE TABLE courses (
  course_id SERIAL PRIMARY KEY,
  teacher_id INTEGER NOT NULL,
  name_course VARCHAR(200) NOT NULL,
  price DECIMAL(10, 2) NOT NULL CHECK (price >= 0),
  FOREIGN KEY (teacher_id) REFERENCES users(user_id) ON DELETE RESTRICT
);
```

CREATE INDEX idx_courses_teacher_id **ON** courses(teacher_id);

CREATE TABLE lessons (
 lesson_id SERIAL **PRIMARY KEY**,
 course_id INTEGER **NOT NULL**,
 name_lesson VARCHAR(200) **NOT NULL**,
 lesson_time INTEGER **CHECK** (lesson_time > 0),
 order_number INTEGER **NOT NULL CHECK** (order_number > 0),
 FOREIGN KEY (course_id) **REFERENCES** courses(course_id) **ON DELETE CASCADE**,
 UNIQUE (course_id, order_number)
);

CREATE INDEX idx_lessons_course_id **ON** lessons(course_id);

CREATE TABLE enrollments (
 enrollment_id SERIAL **PRIMARY KEY**,
 user_id INTEGER **NOT NULL**,
 course_id INTEGER **NOT NULL**,
 enrolled_at TIMESTAMP **DEFAULT** NOW(),
 status VARCHAR(20) **DEFAULT** 'active'
 CHECK (status IN ('active', 'completed')),
 FOREIGN KEY (user_id) **REFERENCES** users(user_id) **ON DELETE CASCADE**,
 FOREIGN KEY (course_id) **REFERENCES** courses(course_id) **ON DELETE CASCADE**,
 UNIQUE (user_id, course_id)
);

CREATE INDEX idx_enrollments_user_id **ON** enrollments(user_id);

CREATE INDEX idx_enrollments_course_id **ON** enrollments(course_id);

CREATE TABLE reviews (
 review_id SERIAL **PRIMARY KEY**,
 user_id INTEGER **NOT NULL**,
 course_id INTEGER **NOT NULL**,
 rating INTEGER **NOT NULL CHECK** (rating **BETWEEN** 1 **AND** 5),
 comment TEXT,
 created_at TIMESTAMP **DEFAULT** NOW(),
 FOREIGN KEY (user_id) **REFERENCES** users(user_id) **ON DELETE CASCADE**,
 FOREIGN KEY (course_id) **REFERENCES** courses(course_id) **ON DELETE CASCADE**,

```
    UNIQUE (user_id, course_id)  
);
```

```
CREATE INDEX idx_reviews_user_id ON reviews(user_id);  
CREATE INDEX idx_reviews_course_id ON reviews(course_id);
```

Задание 4. Частые запросы

1. **«Вывести список всех курсов с именем преподавателя, ценой и общее количество записавшихся студентов».**
2. **«Найти top-5 самых популярных курсов по средней оценке и количеству отзывов».**
3. **«Показать все уроки конкретного курса в правильном порядке их прохождения».**
4. **«Получить историю обучения конкретного студента: список купленных курсов, статус прохождения и оставленные отзывы».**
5. **«Рассчитать общую сумму отчислений за текущий месяц для конкретного преподавателя по всем его созданным курсам».**